GSoC’22 Proposal - CircuitVerse

Project 2 - CircuitVerse Platform Improvements

Personal Details

Name: Vedant Jain
Course:
University:
Current Year:
Email:
Github: https://github.com/vedant-jain03
LinkedIn: https://www.linkedin.com/in/vedant-jain-781006145/
Phone:
Current Country:
Link to Resume:

Table of Content

About Me 3
Motivation and Interest 4
Commitment 5
Contributions so far 5
Proposal 7
- Improvements in Notification Page 7
- Suggested tags for the circuits. 11
- Integrate feature to Sign-in using Gitlab. 14
- Weekly contest 15
- Implement Push Notification (CircuitVerse Mobile App) 19
- SSO (Single Sign-On) Integration 22

Project Plan 24
Task Assignments PRs 28
Additional Information 28

About Me

I am Vedant Jain, a 3rd-year undergraduate student pursuing a Bachelor's degree in Computer Science Branch from Gyan Ganga Institute of Technology and Sciences, Jabalpur, M.P.

I first got familiar with programming in 11th standard, where I got to know about java and had built small projects using Netbeans IDE. It was the phase where I got interested in programming and technology.

Pandemic was one of the most productive times I ever had as I got to know about various things in coding and programming which I don't think I had got in college. For the past year, I explored a lot in open source, web development, problem-solving, and hackathons. The experience I got from each event, contest, or phase taught me a lot of things.

I am a passionate full-stack developer. I have an adequate understanding of various development concepts including git, Github, MongoDB, React, Express, Node, Javascript, Ruby on Rails, C++, and Java. I had built some of my interesting personal projects using MERN stack technology and also demoed one of them at Hacktoberfest.

I am a beta Microsoft learn student ambassador. I have participated in many hackathons and also qualified as a finalist for some of them, especially in the Microsoft Imagine Cup. I am also mentoring open source contributors in Girls-script Summer of Code' 22. I have a 3-star rating in leetcode and CodeChef as well. I participated in Girls-script summer of code 21 as a contributor, here is the link for my contributions. I also tried Google Summer of Code 21 for Chromium Organisation but could not make it.

Motivation and Interest

I have been exploring open source for the past year and have gathered a great interest in it. I try to participate in various global open source programs to gather skills, connections, and knowledge.

I got to know about CircuitVerse from OSD(Open source day) and from then I started my contribution. I started my contribution from a small

SCSS code in the Blog repository of CircuitVerse and now have potentially made PRs in Rails, JS, Jquery, and Jekyll. I never have thought of the contributions I had in CircuitVerse till now when I began and that is because of my interest in working in Web Technology under the guidance of community members.

This project excites me the most as I had previous contributions to Main Platform. I believe that the stated goals of the project can give a better experience to users, easiness to the university, and can engage more user activity in CircuitVerse using the Weekly contest.

Commitment

1. Are you planning any vacations during the GSoC period?

2. How many classes are you taking during the GSoC period?

3. Do you have any other employment during the GSoC period?

4. How many hours per week do you expect to work on the project and what hours do you tend to work?

Contributions so far

PR's submitted

SR	Pull Request	Status	Repository
1	fix: change post method to put method to update notification	Merged	CircuitVerse
2	feat: Send assignment submission report to mentors	Open	CircuitVerse
3	feat: view translation for chinese(zh_CN) language	Open	CircuitVerse
4	feat: integrated tagsinput for project tags	Open	CircuitVerse
5	fix: checkbox not checked for restricted elements while editing	Open	CircuitVerse
6	feat: assignment restriction	Open	CircuitVerse

7	fix: adding group mentor as member	Approved	CircuitVerse
8	feat: added search filter functionality	Open	CircuitVerse
9	feat: adding collaborator via invitation link	Open	CircuitVerse
10	simplifying top navigation	Approved	Interactive-Book
11	fixes: size of group members for 'non-admin group viewer' should be less	Merged	CircuitVerse
12	feat: page progress bar	Merged	Interactive-Book
13	feat: added code for back-to-top button	Open	CircuitVerseDocs
14	feat: added functionality in 'add members in group' like 'add collaborators in project'	Merged	CircuitVerse
15	fixes: swap the position of notifications and testing button	Merged	CircuitVerse
16	feat: added notification button in dropdown-item	Merged	CircuitVerse
17	feat: preserve sidebar collapse state	Merged	Interactive-Book
18	fixes: responsiveness of back-to-top button	Merged	Interactive-Book
19	fixes: converted all the logix to circuitverse views	Merged	CircuitVerse
20	feat: added scroll back to top button	Merged	Interactive-Book
21	fixes: translation of open assignment string using i18n	Merged	CircuitVerse
22	feat: copy to clipboard functionality in invite group link	Open	CircuitVerse
23	feat: convert red color theme to green color theme	Merged	Blog
24	fix: added rel="noopener" attribute to footer external links	Merged	CircuitVerse
25	fix: center items in about us page	Merged	CircuitVerse
26	fix: group_path open_incv redirection feature fixed	Merged	CircuitVerse
27	fixes: Simulator Navbar disappears at the end of tutorials guide	Merged	CircuitVerse
28	Navbar Enhanced	Merged	Blog

Documentation contributions

SR	Pull Request	Status	Repository
1	feat: contribution guideline updates	Merged	CircuitVerse

Other contributions

I have been an active member on CircuitVerse slack channel. I have been a part of the issue-triage team as well. I have supported many new contributors with their development or any sort of queries I can solve. I have also reviewed contributors' PRs.

Proposal

Overview:

This project aims to implement new features and improvements for the CircuitVerse Platform(Mobile and Web) for a better user experience. CircuitVerse Website is written in Ruby on Rails, Javascript, and Jquery. CircuitVerse Mobile app is made by using the Android/Flutter framework.

<u>The goals of the project:</u>

1. Improvements in Notification Page.
2. Implement push notifications for Mobile-app.
3. Implement a new feature- a Weekly contest on the Website.
4. Implement a feature that can be used to suggest tags for the circuits based on the circuit data/description.
5. Integrate SSO(Single Sign-on).
6. Integrate feature to sign in using Gitlab.

<u>Programming Languages to be used:</u>

1. Ruby on Rails.
2. Javascript/Jquery.
3. Flutter-Dart.

1. Improvements in Notification Page

Synopsis

The notification page in the Circuitverse platform serves all

notifications for a user. To achieve this functionality, an activity notification gem is used. Currently, the notifications are being targeted for a user with notifiable events like project forked and project starred. There is also an integration of web push notifications using this gem. But this gem is not being maintained anymore.

For this reason, we need to replace the gem with a more maintained gem and I found noticed gem for this. Currently, the gem supports many notification delivery methods some of which can be very useful in the scope of CircuitVerse like Database, Email, ActionCable channels, Firebase Cloud Messaging.

Before adding a new gem we need to remove the previous gem along with its models, views, controllers, and schema. We can create a new migration file to drop the table of notifications, as **noticed** gem uses the table name "notifications" as default that will have different attributes. By doing this, there will be no database collision between gems functionality *(I will refer to this PR where integration is done for activity_notification)*.

Improvements we can do

1. Adding notifications for
   - Author, if someone starred in a project.
   - Author, if someone forked a project.
   - Group members on new assignments published by group mentors(new).
   - Group members on assignment deadline(new).
   - Group members are invited by a group mentor(new).
2. Adding tags for notifications based on their type. Example: ***project, assignment, group.***
3. Improvements in the user interface (UI).
4. We can also have different tabs where we can render notifications for a specific tag. For example, in the project tab, all notifications with the tag "Project" will render. The inbox tab will contain all notifications.
5. We can add another section for unread notifications on the same page.

Installation and initial setup for 'noticed' gem

After successfully removing the activity_notification gem, we can start setup **noticed** gem by the following steps:

- Add it to Gemfile by running `bundle add "noticed"`
- To save notifications to the database, we need to generate a Notification model.

  ```
  rails generate noticed:model
  ```

  This will generate a Notification model and instructions for associating User models with the notifications table. After running migration we expect to see changes in the schema.rb about notification table. ("noticed" gem uses "notifications" table name as default).
- We need to modify the relation between the user and notification model.
  - In model/user.rb

    ```
    has_many :notifications, as: :recipient
    ```
  - In mode/notification.rb

    ```
    class Notification < ApplicationRecord
     include Noticed::Model
     belongs_to :recipient, polymorphic: true
    end
    ```

Implementation for database Delivery method(User Notifications)

For the implementation of notification for events, I would be taking the example of "Notify Author when someone stars a project".

1. We need to generate a notification class.

   ```
   rails generate noticed:notification StarNotification
   ```

2. We need to modify the member functions and data members according to the notification we want to save and render.

   ```
   class StarNotification < Noticed::Base
    # Add your delivery methods
   ```

```
  #
  deliver_by :database, format: :to_database

  # Add required params
  #
  param :project
  def to_database
    {
      type: self.class.name,
      params: params,
      account: Current.account
    }
  end

  # Define helper methods to make rendering easier.
  #
  def message
    "Someone starred in your Project."
  end

  def url
    @project = Project.friendly.find(params[:project])
    user_project_path(@project.author_id, @project)
  end
end
```

3. Finally, we need to create a new object for StarNotification and call the delivery method type we wanted.

   In controller/project_controller.rb, we need to call it in the star toggle method.

```
notification = StarNotification.with(project: @project.id)
notification.deliver(@project.author_id)
```

In a similar fashion we can generate a notification for all events.

For rendering the notifications in view, we need to generate a controller '**Notification controller'** which will inherit from **Application Controller** in which we can declare an instance variable.

```
class NotificationsController < ApplicationController
   before_action :authenticate_user!

   def index
       @notifications = current_user.notifications
```

```
    end
end
```

We need to configure a route for the notification page as well in routes.rb.

```
get "/notifications", to: "notifications#index", as: "notification_path"
```

Finally we can render all notifications in view file.

```
<h1>Notifications</h1>
<% @notifications.each do |notification| %>
   <span><%= notification.message %></span>
<% end %>
```

We can render different notification on different tabs like Project, Assignment, Group etc. by adding a params for every notification and then filter every notification with (notification.tag == #{current_tab})? in each tab.

To render unread notifications, noticed have an instance method @notification.unread?. Using this method we can filter for each @notifications and if it is true then render the notification otherwise not.

Noticed have an attribute in a table called read_at if it is nil that means the notification is unread and every time a user visits the notification page we need to make it read in the controller using class method `user.notifications.mark_as_unread!.`

Key Deliverables

- Integrate the noticed gem by replacing activity_notification gem.
- Notification page UI revamp.
- Initialize notification events for the *'star a Project'* and *'forked the project'*, as these events are initialized currently using activity_notification gem.

UI Revamp

- Sample improved UI.

2. Suggested tags for the circuits.

Synopsis

Currently in CircuitVerse a user can add tags to a Project without any suggestion. By implementing this feature, a dropdown will popup after a user inputs something in the textfield(tag_list), dropdown will contain a list of options having suggested value based on the project data and description that can be referred to be a tag.

Implementation and steps

- The primary task is to get the valid suggested tags based on project data, we can implement the following filters and steps to generate a valid array of suggested tags in project_controller.rb:

```
# method to create suggested tags
def create_suggested_tags
   # 1. predefined suggest tags on the basis of metadata.json elements
   @suggested_tag =
["Input","Output","NotGate","OrGate","AndGate","NorGate","NandGate","XorGate","
XnorGate","SevenSegDisplay","SixteenSegDisplay","HexDisplay","Multiplexer","Bit
Selector","Splitter","Power","Ground","ConstantVal","ControlledInverter","TriSt
ate","Adder","verilogMultiplier","verilogDivider","verilogPower","verilogShiftL
eft","TwoComplement","verilogShiftRight","Rom","RAM","verilogRAM","EEPROM","Tfl
ipFlop","JKflipFlop","SRflipFlop","DflipFlop","TTY","Keyboard","Clock","Digital
Led","Stepper","VariableLed","RGBLed","SquareRGBLed","RGBLedMatrix","Button","D
emultiplexer","Buffer","SubCircuit","Flag","MSB","LSB","PriorityEncoder","Tunne
l","ALU","Decoder","Random","Counter","Dlatch","TB_Input","TB_Output","ForceGat
e"];

   # 2. using project description for tags
   @project = Project.friendly.find(params[:id])
   description = @project.description
   @description_tags = description.split(" ");

   # 3. using project name for tags
   name = @project.name
   @name_tags = name.split(" ");

   # 4. As for project description we use trumbowyg, after receiving the data
we might get some unexpected HTML tags. To overcome this we need to extract the
text from every string using regex expression
   valid_tag_regex = /(<([^>]+)>)/i
   @description_tags.each do |tag|
```

```
    tag.gsub! valid_tag_regex , ''
  end

  # 5. some of invalid tags
  @invalid_tags = ["the", "will", "be", "only", "if", "any", "or", "all",
"are", "a", "You", "from"]

  # 6. combining set of different tags
  @final_tags = @suggested_tag + @description_tags + @name_tags -
@invalid_tags

  # 7. making the final tags array unique to avoid duplicacy
  @final_tags = @final_tags.uniq

  # 8. passing it to set_project
  session[:passed_variable] = @final_tags
 end
...

  # Use callbacks to share common setup or constraints between actions.
  def set_project
    @suggested_tag = session[:passed_variable];
    ...
  end
```

- After we get the valid tags array, we can use select2 to render the Automatic suggestion dropdown feature for our tag textfield.

  Implementation:

  1. Text field of tag_list

```
        <%=  form.text_field  :tag_list,  class:  'form-control  form-input
js-example-basic-example', multiple: "multiple", id: "tag_list"  %>
```

  2. We need to generate array of objects which contain {id, final_data_tags[id]} to pass to select2 plugin.

```
# Retrieve tags from backend
 var final_data_tags = <%= raw @suggested_tag %>;
 var data = [];
 var len = final_data_tags.length;

# converting it into select2 format dataset
 for (var i = 0; i < len; i++) {
 data.push({
     id: i,
     text: final_data_tags[i]
```

```
});
}
var area_Ref = $('.js-example-basic-example');
area_Ref.select2({
  data: data
})
console.log(data);
$(document).ready(function() {
  area_Ref.select2({
    placeholder: 'Enter some tags',
    tags: true,
    tokenSeparators: [',', ' ']
  });
});
```

Result

Key Deliverables

- To display the suggested tags on the basis of Project Description and name.

3. Integrate feature to Sign-in using Gitlab.

Synopsis

Currently in Circuitverse we have a Sign-in feature using email-password, GitHub, Google, and Facebook.

Implementing the Sign-in feature using Gitlab can enhance the scope for users to use CircuitVerse.

Implementation and steps

- We will be using omniauth gitlab gem, as similar, as we did for other third-party authentication.
- We need to configure **GITLAB_CLIENT_ID** and **GITLAB_CLIENT_SECRET** in this file.
- Add SVG file to render gitlab icon for app/views/users/shared/_links.html.erb file.
- Configure config/initializers/devise.rb

```
config.omniauth :gitlab, ENV['GITLAB_CLIENT_ID'],
ENV['GITLAB_CLIENT_SECRET'], {
   :scope => 'read:user'
 }
```

- Add configuration for gitlab in app/models/user.rb.

```
devise :database_authenticatable, :registerable, :recoverable,
:rememberable, :trackable,
        :validatable, :omniauthable, omniauth_providers: %i[google_oauth2
facebook github gitlab]
```

- Modification in controllers/users/omniauth_callbacks_controller.rb

```
def gitlab
   generic_callback("gitlab")
 end
```

- Modification in app/views/users/shared/_links.html.erb

```
<a href="/users/auth/gitlab">
            <%= image_tag "SVGs/gitlab.svg",alt: "GitlabIcon", :class =>
"users-social-links" %>
     </a>
```

Key Deliverables

- Integrate feature for GitLab SignIn.

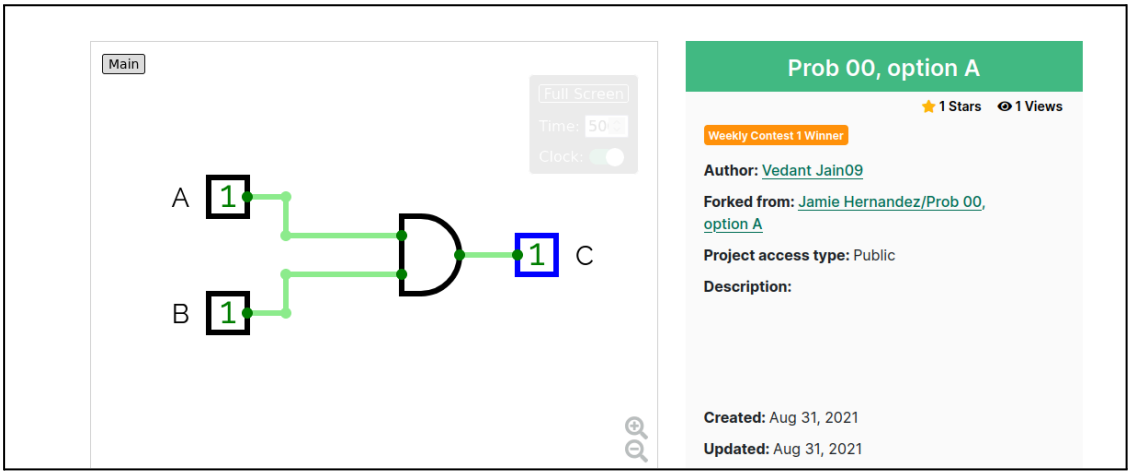
4. Weekly contest

Synopsis

Weekly contests on the CircuitVerse will be open for every user. Users can register in contests and can submit one of their creations or Project on the CircuitVerse platform. The top projects and project authors will be the winner. The judging criteria for projects will be the votes by other users. The more the votes, the higher will be the priority.

- The contest can be scheduled automated using ActiveJobs implementation in Rails. Every contest will be scheduled on Sunday and will end on Saturday(6 days).

- Users can register for only one project in a contest.
- The winning project will get featured on the **“Featured Circuits”** page. We can have a golden tag for winning projects with a tag message as **“Weekly Contest Winner #{contest_number}”.**



- We can add an attribute in the user’s profile of “**Weekly Contest Won”.** For every winning project, the project’s author will get an increment in this attribute.
- Some changes to avoid Plagiarism in the Contest:
    - For a span of ending time of the contest, a registered project will not have an option of **Copy or Forked**.
    - A project that a user wanted to register for a contest should be their own project, not a forked one(Slug).
- We can use the announcement feature of CircuitVerse to notify all the visiting users about the Weekly Contest.
- The circuit or project the user wants to register should be public.

Implementation with pseudo code

1. Set up the schema.
   - We need to create two tables contest and submissions with the following attributes:
     a. Contest

```
create_table "contests", force: :cascade do |t|
  t.jsonb "winners", default: []
  t.datetime "start_date"
  t.datetime "end_date"
  t.datetime "created_at", precision: 6, null: false
  t.datetime "updated_at", precision: 6, null: false
```

```
end
```

b. Submissions

```
create_table "submissions", force: :cascade do |t|
  t.bigint "contest_id"
  t.bigint "user_id"
  t.bigint "project_id"
  t.bigint "votes"
  t.datetime "created_at", null: false
  t.datetime "updated_at", null: false
  t.index ["contest_id"], name: "index_submissions_on_contest_id"
  t.index ["user_id"], name: "index_submissions_on_user_id"
  t.index ["project_id"], name: "index_submissions_on_project_id"
end
```

- We need to set up key relations between models.

```
add_foreign_key "submissions", "contests"
add_foreign_key "submissions", "users"
add_foreign_key "projects", "submissions"
```

- We need to modify user table

```
  t.bigint "contest_won"
```

- We need to modify project table with addition of 2 more

```
  t.bigint "contest_id"
  t.boolean "contest_submission", default: false
```

2. Adding Association between models.

- models/contest.rb

```
has_many :submissions, dependent: :destroy
```

- models/submission.rb

```
belongs_to :contest
belongs_to :user
has_one :project
```

- models/user.rb

```
has_many :submissions, dependent: :destroy
has_many :contests, through: :submissions
```

- models/project.rb

```
belongs_to :submission
```

3. Modification in Controllers.

- controllers/contest.rb

```
# creating new contest

def show
  @submissions = @contest.submisions
end

def new_submission
  @project = Project.find(params[:project_id])
  if @project.contest_submission?
    notice = "You have already registered your circuit."
  elsif @project.contest_id?
    notice = "This circuit have already won, try registering another one."
  else
    current_user.submission.create!(contest: @contest)
    notice = "Your submission successfully added in the Contest."
  end
  redirect_to contest_page_path(@contest), notice: notice
end
```

- controllers/submission.rb

```
def create
  @submission = @contest.submission.new(submission_params)
  @project = Project.find(submission_params[:project_id])
  @project.contest_submission = true
  @submission.votes = 0
  @submission.save
end

private

def submission_params
  params.require(:submission).permit(:user_id, :project_id, :contest_id)
end
```

4. Active Jobs so that we can automate the flow of contests every week.

- Jobs/contest_job.rb

```
# pseudo code
```

```
class ContestJob < ApplicationJob
 queue_as :default
  # recurrence
 after_perform do |job|
   self.class.set(:wait => 1.week).perform_later
 end
 def perform
   # listing current contest
   current_contest = Contest.order("created_at").last
   # all submissions of current contest
   submissions = current_contest.submissions
   # sorting according to votes with limit 3
   winning_submissions = submissions.find(:all, :order => "votes", :limit =>
3)
   # awarding winning submissions
   winning_submissions.each do |subm|
     project = Project.find_by(subm.project_id)
     # adding project to featured circuit
     featured_circuit = FeatureCircuit.new
     featured_circuit.project_id = project.id
     featured_circuit.save!
     # awarding user with increment in contest won field
     user = User.find_by(subm.user_id)
     update_contest_won = user.contest_won + 1
     user.update(contest_won: update_contest_won)
   end

   # scheduling new contest
   newcontest = Contest.new
   newcontest.start_date = Time.zone.now
   newcontest.end_date = Time.zone.now + 1.week
   newcontest.save!
   # Announcement for new contest
   announcement = Announcement.new
   announcement.body = "New Weekly Contest is here, check it!"
   announcement.link = contest_page_url
   announcement.start_date = Time.zone.now
   announcement.end_date = Time.zone.now + 5.day
   announcement.save!
 end
end
```

UI Samples

- Contest Page.

Key Deliverables

- Automation of scheduling weekly contests using ActiveJobs.
- To Reward the winning project by displaying them on Feature Projects Page.

In addition

- We can create a dashboard to handle it manually as well.

5. Implement Push Notification (CircuitVerse Mobile App)

Synopsis

CircuitVerse Mobile App is written in Flutter. Flutter provides a rich set of built-in default widgets along with the creation of flexible custom widgets.

To integrate Push Notification in Mobile App, we can use FCM.

We can integrate FCM with flutter, the other advantage to doing so is both are powered by Google.

But how can we send the Main website notification to Mobile App?

This can be solved using noticed gem as it has FCM delivery methods. For every notification controller, we can send a web notification as a message to the Firebase app using the FCM delivery method and FCM will push that message to the Mobile app as a notification hence a user will get a push notification in their respective mobile device.

Implementation

1. Add new dependencies.

   In mobile-app/pubspec.yaml

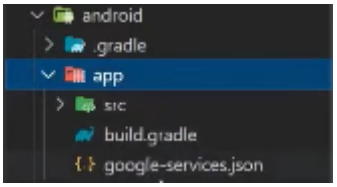
   - firebase_core (used to connect Flutter app and firebase)
   - firebase_messaging (use to get the firebase messages to

flutter app)
- flutter_local_notifications (use to display notifications)

2. We need to configure a firebase project.

   Make a new project and register the app:

   a. Go To Console.
   b. Add project.
   c. We need to register a new android app for our project with a package name field filled with *"org.circuitverse.mobile_app".*
   d. We need to download the google-service.json file provided during the process of registering the app.



   e. We need to add configuration in android/build.gradle

```
buildscript {
  dependencies {
    ...
    // Add this line
    classpath 'com.google.gms:google-services:4.3.10'
  }
}
```

   f. We need to apply the plugin and add dependency in the android/app/build.gradle file.

```
apply plugin: 'com.google.gms.google-services'

dependencies {
  ...
  implementation platform('com.google.firebase:firebase-bom:29.2.1')
}
```

3. We need to run `flutter pub get` to reflect changes.

   We can now send notifications from Cloud Messaging to our registered page.

4. Connectivity of Rails App (Website) and FCM.

   We need to configure the private key so that rails can modify the

firebase project. We need to download the key configuration file that will look something like this. This JSON file will contain the necessary credentials to send notifications via Google Firebase Cloud Messaging.

5. We can now set up the delivery method and notification controller.

```
# Example Pseudo code

class StarNotification
  deliver_by :fcm, credentials: :fcm_credentials, format:
:format_notification

  # This needs to return the path to your FCM credentials
  def fcm_credentials
      Rails.root.join("config/certs/fcm.json")
  end

  def format_notification(device_token)
      {
      token: device_token,
      notification: {
      title: "Project name",
      body: "Someone starred in your project."
      }
      }
  end
end
```

Alternatives to Firebase with notifications as core-feature

- Parse
- Back4app
- AWS-amplify
- Backendless

Alternate solution for Push Notification

The other solution is using Rails API. We can create an endpoint in our Rails API and we can periodically call it for new notifications that are unread.

But the question is, when the app is not active or when the app is in the background, how can we call the API?
It can be done using the workmanager library that is used to do

background fetching, but the only reason that it is not better is because of the cooldown period. It takes 15 minutes to request another API call in the background.

Key Deliverables

- To implement Push Notification for CircuitVerse Mobile App.

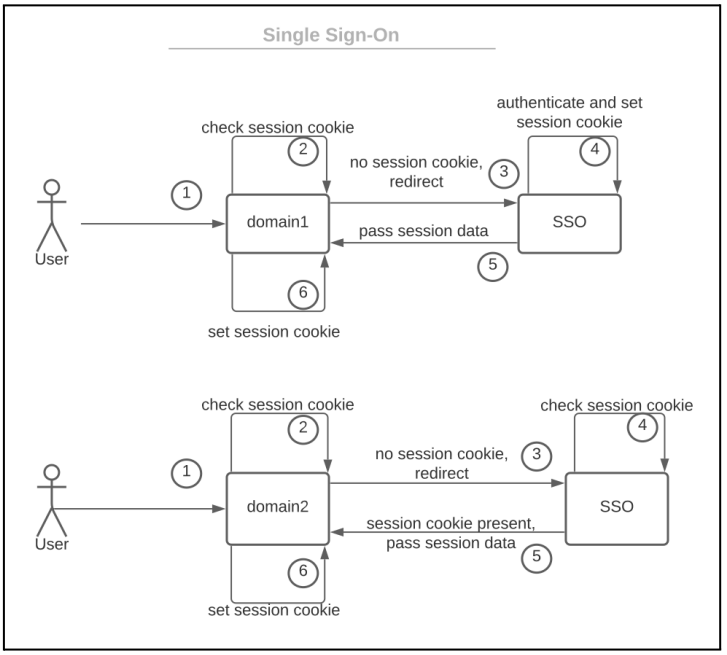
6. SSO (Single Sign-On) Integration

Synopsis

Single sign-on (SSO) is a session and user authentication service that permits a user to use one set of login credentials, for example, a username and password to access multiple applications. Many institutions that use CircuitVerse use a local copy of CircuitVerse with a custom SSO authentication. It will make tasks easy for institutes by natively supporting SSO authentication in CircuitVerse.

Implementation

We can use SAML SSO. SAML SSO works by transferring the user's identity from the identity provider to the service provider.



1. We need to add a new dependency for it.

```
gem 'devise_saml_authenticatable'
```

2. Add saml_authenticatable to user model user.rb

```
devise :saml_authenticatable
```

3. Add the following to your config/initializers/devise.rb file:

```
Devise.setup do |config|
      config.saml_create_user = true
      config.saml_update_user = true
      config.saml_default_user_key = :email
      config.saml_session_index_key = :session_index
      config.saml_use_subject = true
      config.idp_settings_adapter = nil
      config.saml_configure do |settings|
        settings.assertion_consumer_service_url =
"#{$callback}/users/saml/auth"
        settings.assertion_consumer_service_binding =
"urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
        settings.name_identifier_format             =
"urn:oasis:names:tc:SAML:2.0:nameid-format:transient"
        settings.issuer                             =
"#{$callback}/users/saml/metadata"
        settings.authn_context                      = ""
        settings.idp_slo_target_url                 = ""
        settings.idp_sso_target_url                 = "IDP URL"
        settings.idp_cert_fingerprint               = 'IDP_CERTIFICATE'
        settings.idp_cert_fingerprint_algorithm     =
'http://www.w3.org/2000/09/xmldsig#sha256'
      end
    end
```

Where callback will be:

```
$callback = Rails.env.development? ? 'http://localhost:3000' :
ENV['CALLBACK_ADDRESS']
```

4. Set up IDP(Identity Provider)
5. We need to create a new file in config folder called attribute-map.yml with content:

```
  "urn:mace:dir:attribute-def:uid": "name"
```

```
"urn:mace:dir:attribute-def:email": "email"
```

This tells Devise SAML what attributes returned from Id Provider map to which attribute of User model.

In our devise.rb we have configured Devise SAML as such so it creates new users in the app if they have not been seen before. It also updates the mapped attributes when they change on the Id Provider side. These are configured via saml_create_user and saml_update_user parameters.

Key Deliverables

- SSO integration.
- Documenting the steps in CircuitVerse Docs.

Project Plan

The major part will be to implement the Weekly Contest feature and replace & implement noticed gem on the main platform. Push notification should be done after initiating noticed gem in rails so it can be done after setup of noticed gem in the main platform.

In the pre-GSoC period, I will start learning more concepts of Rails to deliver the best-structured code and features and also will build UI for notification and weekly contest pages. I will learn dart-flutter as well for the Mobile-App Push Notification implementation.

In the 1st phase, I will start with the implementation of Gitlab authentication and then SSO integration, as both lie at the core of authentication. After that, I will start the implementation of the Weekly contest feature for the Main platform.

In the 2nd phase, I will start with the suggested tags feature for the Project page. After that, I will jump into the integration of Noticed gem and will finalise the UI for the notification page. Then, finally implementation of Push Notifications in Mobile-app.

I will be learning and coding throughout the week and will be reporting

my work every weekend to mentors and will ask any doubts in between.

Project Plan - Preliminary Plan

Community Bonding Period [May 20 - June 12]

The GSOC period is about 12 weeks with about 2 weeks of community bonding.

Before and During the community bonding I will gather more detailed requirements about the features to be implemented and document the acceptance criteria so that I can serve as a guideline while development and testing. Requirement gathering includes documenting the details design of the app's UI and how the different features should be implemented.

The detailed timeline is shown below. I will spend 70% of the allocated time to complete each of the deliverables and use the remaining 30% for testing and improvement to be made after feedback is gotten on each of the deliverables by mentors.

Week number	Start Date	End Date	Tasks to be completed
Phase 1	**June 13**	**29 July**	
Week 1	June 13	June 19	- Add a feature to sign-in using Gitlab along with its configuration. - Pull request for Gitlab sign-in feature. - Work on request changes(if any). - Publish the blog.
Week 2	June 20	June 26	- Add integration of SSO (Single Sign-On) - Documenting SSO integration for Universities for easiness and clear idea. - Pull request for SSO integration. - Work on requested changes(if any). - Publish the blog

Week number	Start Date	End Date	Tasks to be completed
Week 3 - Week 4 - Week 5	27 June	17 July	- Start working on UI for Weekly contest - Review UI from mentors - Integration of MVC for weekly contest, setup of schema, and ActiveJobs. - Make a Draft PR for implementation. - Work on request changes(if any). - Start working on Manual Dashboard (UI + Integration). - Work on requested changes(if any). - Will make a final PR.
Week 6	18 July	24 July	- Will write test cases for the weekly contest. - Major Improvements before the phase one evaluation. - Publish the blog (Weekly contest).
Phase 1 Evaluation	25 July	29 July	**Key Deliverables:** - Gitlab Sign-in feature Integration - SSO integration - Weekly Contest Feature Integration.
Phase 2	30 July	4 September	
Week 1	30 July	6 August	- Start working on the suggested tags feature. - UI review by mentors. - Test cases. - Final PR - Work on requested changes(if any) - Publish a blog - Start learning about noticed gem (or any other suggested by mentors)
Week 2 - Week 3	7 August	20 August	- Will remove current gem. - Make a draft PR.

Week number	Start Date	End Date	Tasks to be completed
			- Start working on UI for notification. - Setup of new gem (noticed) for notifications. - Integration of notification controllers, views, models, and schema setup. - PR and will work on requested changes(if any). - Publish a blog
Week 4	21 August	26 August	- Will configure Firebase project for FCM - Start setup of FCM delivery method in a website backend. - Will start integration of push notification in Mobile-App (Flutter). - PR and will work on requested changes(if any).
Week 5	27 August	4 September	- Major Improvements before final evaluation. - Publish a blog.
Final Evaluation	5 September	12 September	**Key Deliverables:** - Suggested tags for Projects. - Notification Improvement(Main Platform) - Push Notification for Mobile App.

Testing and Verification

At the end of each milestone above. The new feature will be tested with respect to the requirement specifications documented earlier as issues on Github. Feedback will be received from Mentors and fellow GSOC students. Changes proposed in the feedback will be implemented until full satisfaction of the mentor.

Code Review is also a great way to ensure that all cases in requirement are covered, no logical error exists and that code conforms to the existing style guidelines.

Major Milestone

1. Weekly Contest Feature Integration.
2. Remove the current gem and add a new gem along with integration in Notifications for Main Platform.
3. Implement Push Notification in Mobile App.

Task Assignments PRs

- Setup Project [Done].
- Add specs to existing code [Done].
- Translate one page to your native language using i18n [Done].
- Email verification during signup using Devise [Done].

Additional Information

OS: Dual Boot (Ubuntu and Windows)
Code editor: VS code

I believe that I am the perfect candidate to complete the project as my major contributions are based on main platform improvement(Rails). As a result of my contributions, I am familiar with the major parts of the codebase and have a good understanding of the codebase.

I have done most of the project research by implementing some features in localhost as well, and after getting an overview and concept I have put them into the proposal.

The CircuitVerse community has given me a great experience, I have learned a lot from mentors and contributors. I am very passionate to work with CircuitVerse in the future.

Mentors

Ayan Biswas, Samiran Konwar, Shivansh Srivastava, Pavan Joshi, Aboobacker MK